Task 3 实验报告

Task 3 实验报告
- 1 测试环境与效果展示
- 2 实现框架
  - 2.1 Tensor 类与 Function 类
  - 2.2 算子抽象层级设计
    - 2.2.1 算子类层
    - 2.2.2 张量函数层
    - 2.2.3 指针函数层
    - 2.2.4 CUDA 层
    - 2.2.5 核函数层
  - 2.3 计算图与自动梯度
  - 2.4 Python 中的算子封装
  - 2.5 Adam 优化器
- 3 性能优化
  - 3.1 初始化责任划分
  - 3.2 异步显存分配
  - 3.3 Tensor 缓存池
- 4 实验与数据分析
  - 4.1 实验设计思路
  - 4.2 Cifar10
  - 4.3 Tiny ImageNet
- 5 总结
- A 附录
  - A.1 实验细节与复现方式
  - A.2 Cifar10（MyTensor）
  - A.3 Cifar10（PyTorch）
  - A.4 Tiny ImageNet（MyTensor）
  - A.5 Tiny ImageNet（PyTorch）

1 测试环境与效果展示

- 设备参数：
  - GPU: RTX 4090(24GB) * 1
  - CPU: 16 vCPU Intel(R) Xeon(R) Platinum 8358P CPU @ 2.60GHz
  - PyTorch 2.8.0
  - Python 3.12(ubuntu22.04)
  - CUDA 12.8

结果展示（Cifar10）	MyTensor	PyTorch	与 PyTorch 的比值
参数量	1 188 170	1 188 170	100%
训练效率 / epoch	9.03s	9.70s	93.1%
最佳测试集准确率	89.87%	90.14%	99.7%

结果展示（Tiny ImageNet）	MyTensor	PyTorch	与 PyTorch 的比值
参数量	13 283 848	13 283 848	100%
训练效率 / epoch	49.07s	30.23s	162%
最佳验证集准确率	41.37%	42.69%	96.9%

注:

- 统计 epoch 耗时时，包含所有数据加载、前向、反向与参数更新时间，但不包含测试集评估时间。
- Tiny ImageNet 未公布测试集标注，所以仅在验证集上评测准确率。

2 实现框架

2.1 Tensor 类与 Function 类

Tensor 类是我们实现的核心数据结构，负责存储多维数组数据及其梯度信息。Function 类则是所有算子的基类，负责定义前向计算和反向传播的接口。每个具体的算子（如加法、卷积等）都继承自 Function 类，实现其特定的前向和反向逻辑。

我们的 Tensor 类定义如下示例:

```cpp
// in tensor.h

typedef std::vector<size_t> shape_t;
struct Tensor;
struct Function;

struct Tensor {
    std::shared_ptr<float[]> data, grad;
    shape_t shape;
    size_t size;
    bool useGrad;
    std::shared_ptr<Function> creator;
};

struct Function {
    const std::string name;
    Function(const std::string& name): name(name) {}
    virtual Tensor forward(const Tensor&) {
        throw std::runtime_error("forward(input) not implemented");
    }
    /* ... */
};
```

- `data` 存储张量的数据;
- `grad` 存储梯度，在 `useGrad == false` 时为 `nullptr`;
- `shape` 定义张量的形状;
- `size` 是元素总数;
- `useGrad` 标志是否需要计算梯度;
- `creator` 指向生成该张量的算子，若此张量为叶子节点则为 `nullptr`。

由于在此项目中我们总将 Tensor 定义在 GPU 上，因此我们省略了对 Tensor 所在设备的记录，并假定 `data` 和 `grad` 均为 GPU 内存指针。此外，我们没有对 Tensor 进行 view 或 slice 等操作，因此也省略了诸如 `stride` 等属性。

我们使用 `std::shared_ptr` 来管理 `data` 和 `grad` 的内存，以确保在多个 Tensor 共享同一数据时不会发生内存泄漏或悬空指针的问题。特别地，我们使用 `data` 的地址（作为一个 64 位整数）来唯一标识一个 Tensor，从而在计算图中追踪张量的来源和去向，这也得益于我们不需要对 Tensor 进行 view 或 slice 操作，否则需要更复杂的标识机制。

2.2 算子抽象层级设计

为了提升代码的可维护性和扩展性，尤其是避免过多的对 CUDA 的底层调用，我们设计了多层次的算子抽象：

2.2.1 算子类层

本层定义所有算子的类接口，包括前向计算和反向传播方法。每个具体的算子类都继承自 Function 类，具有 `forward` 和 `backward` 方法，并自行保存所需的中间变量。特别地，如果某个算子没有内部参数，我们也额外实现了它的函数式接口，方便用户直接调用而无须创建一个算子对象。例如，我们实现了如下 `relu` 函数。它通过 `std::make_shared` 创建了一个 ReLU 算子对象，以此保证对此对象的引用计数正确，且此对象的生命周期至少与返回的 Tensor 一样长。

```
// in operator.cpp

Tensor relu(const Tensor& input) {
    return std::make_shared<ReLU>()->forward(input);
}
```

特别地，我们在这一层会检查输入张量的 `useGrad` 标志，并决定输出张量的 `useGrad` 标志。如果输入张量的 `useGrad == false`，我们就认为这是一次 Inference 计算，因此输出张量的 `useGrad` 也应为 `false`，且不进行计算图构建或反向传播的任何准备工作，从而避免不必要的梯度计算和内存开销。诸如 Dropout 这类在 Inference 时行为不同的算子，也会在这一层进行特殊处理。

2.2.2 张量函数层

本层定义所有算子的函数接口，负责对 Tensor 类的调用封装。它们通过 Tensor 类传递输入与输出，检查各 Tensor 的形状，并正确的设置低层次函数的调用参数。它们不进行任何实际的计算调用。这一层封装保证了内存操作的安全性。例如，我们实现了如下 `forward_conv2d` 函数，它只在启用调试时进行输入形状检查，并以正确的形状参数调用指针函数层的函数。

```
// in functional.cu

void forward_conv2d(const Tensor& X, Tensor& Y,
                    const Tensor& Ker, const Tensor& b) {
#ifdef SHAPE_CHECK
    assert(X.shape.size() == 4);
    assert(X.shape[0] == Y.shape[0]);
    assert(X.shape[1] == Y.shape[1]);
    assert(X.shape[2] == Y.shape[2]);
    assert(Ker.shape.size() == 4);
    assert(Ker.shape[0] == Y.shape[3]);
    assert(Ker.shape[1] == X.shape[3]);
    assert(Ker.shape[2] == Ker.shape[3]);
```

```
    assert(Ker.shape[2] % 2 == 1);
    assert(b.shape.size() == 1);
    assert(b.shape[0] == Ker.shape[0]);
#endif
    int C_in = X.shape.back();
    int W = X.shape[X.shape.size() - 2];
    int H = X.shape[X.shape.size() - 3];
    int N = X.size / (H * W * C_in);
    int C_out = Ker.shape[0];
    int K = Ker.shape[2];
    _forward_conv2d(X.data.get(), Y.data.get(),
                    Ker.data.get(), b.data.get(),
                    N, H, W, C_in, C_out, K);
}
```

2.2.3 指针函数层

本层定义所有算子的数据（`data` 或 `grad`）指针调用接口。它们接收原始的指针参数（如 `float*`）和必要的形状信息，进行 CUDA 层的调度。从这一层次起，我们无法也不必再检查张量形状，而总在存储级别将它们视作一维数组进行处理。例如，我们实现了 `_backward_fc` 函数，它调用 `GpuFunc` 命名空间下的 CUDA 层函数完成了反向传播计算。

```
// in functional.cu

void _backward_fc(const float* X, const float* Y, const float* W,
                  const float* b, int N, int C_in, int C_out,
                  const float* dY, float* dX, float* dW, float* db) {
    GpuFunc::gemm(dY, W, dX, N, C_in, C_out,
                  CUBLAS_OP_N, CUBLAS_OP_N, 1.0f, 1.0f);
    GpuFunc::gemm(dY, X, dW, C_out, C_in, N,
                  CUBLAS_OP_T, CUBLAS_OP_N, 1.0f, 1.0f);
    GpuFunc::colSum(db, dY, C_out, N * C_out);
}
```

2.2.4 CUDA 层

本层定义必要的 CUDA 函数实现。它们负责调度 GPU 计算资源，正确配置线程参数并启动核函数，或者调用 CUBLAS 等库函数进行计算或配置。例如，我们实现了 `gemm` 函数，它调用 CUBLAS 库完成矩阵乘法计算，但将其封装为更易用的行主序形式。

```
// in cudabase.cu

void gemm(const float* A, const float* B, float* C, int M, int N, int K,
          cublasOperation_t transA, cublasOperation_t transB,
          float alpha, float beta) {
    // C^T = alpha * opB(B^T) * opA(A^T) + beta * C^T
    // C just represents C^T in cublas's view
    int ldb = (transB == CUBLAS_OP_N) ? N : K;
    int lda = (transA == CUBLAS_OP_N) ? K : M;
    cublasSgemm(_handle, transB, transA,
                N, M, K,
                &alpha, B, ldb, A, lda,
                &beta, C, N);
```

```
}
```

2.2.5 核函数层

本层定义必要 CUDA 核函数。它们负责具体的并行计算逻辑实现。由于启动线程的数量与实现逻辑强绑定，它们只应该由对应的 CUDA 层函数负责调用。例如，我们实现了 `_broadcast` 函数，抽象了不同的张量广播需求并实现了统一的核函数。

```cpp
// in cudabase.cu

__global__ void _broadcast(float* dst, const float* src,
                           int N, int M, int C) {
    CUDA_KERNEL_LOOP(i, N) {
        dst[i] = src[i % (C * M) / C];
    }
}
```

2.3 计算图与自动梯度

我们通过在 Tensor 类中保存 `creator` 指针来构建计算图。在前向计算过程中，每当一个新的 Tensor 被创建时，我们将其 `creator` 指向生成它的算子对象。这样，整个计算过程就形成了一个有向无环图（DAG）。值得注意的是，在我们的抽象方式中，Tensor 和 Function 对象都是 DAG 中的结点。对于 Tensor 结点，它要不是叶子，要不由 `creator` 指针记录了唯一的入边；对于 Function 结点，它的入边来自其输入 Tensor（记录在子类的 `in` 属性中，可由 `parent()` 方法获取），出边指向其输出 Tensor（记录在此 Tensor 的 `creator` 中）。

在反向传播过程中，首先，我们从输出 Tensor 开始，沿着 `creator` 和 `parent` 追踪计算图，维护一个以 `data` 地址值为 key，对应 Tensor 在反向 DAG 的入度为 value 的 Hash 表（实现时为了稳定性采用的是 `std::map` 平衡树）。接着，我们开始在反向 DAG 上进行拓扑排序遍历，得到拓扑序列。最终，我们依照 Tensor 的拓扑顺序调用相应 `creator` 的 `backward` 方法计算梯度。这样，我们确保了每个 Tensor 的梯度在其所有子节点的梯度计算完成后才被计算出来，符合链式法则的要求。

2.4 Python 中的算子封装

如 2.2 节所介绍的，我们在 C++/CUDA 层实现了所有必要的算子逻辑，并通过 Pybind11 将它们暴露给 Python。在 Python 层的算子类，我们只需要创建算子所需的参数 Tensor，并在 `forward` 方法中调用 C++ 层的函数式接口完成计算。这样，我们就实现了 Python 层算子的封装，方便用户在 Python 中使用这些算子构建模型和进行训练。例如，我们实现了 `Linear` 算子类：

```python
class Linear:
    def __init__(self, in_dim, out_dim):
        self.weight = mt.Tensor([out_dim, in_dim], True)
        self.weight.uniform((2.0 / in_dim)**0.5)
        self.bias = mt.Tensor([out_dim], True)
        self.bias.zero_data()

    def __call__(self, x):
        return mt.linear(x, self.weight, self.bias)

    @property
    def parameters(self):
        return [self.weight, self.bias]
```

2.5 Adam 优化器

为了方便用户注册和使用优化器，我们决定在 Python 中实现 Adam 优化器本体逻辑。用户只需将模型参数的 Tensor 列表传递给 Adam 优化器对象，并在每次迭代后调用 `step()` 方法即可完成参数更新。Adam 优化器会自动维护每个参数的动量和二阶动量估计，并根据设定的学习率和正则化参数进行更新。

对于优化器的计算过程，则由 MyTensor module 中的 `adamStep` 等函数完成。它们接收参数 Tensor 的数据和梯度指针，进行必要的计算和更新。实现逻辑与 2.2 节中描述的多层次算子设计类似，不再赘述。

3 性能优化

3.1 初始化责任划分

这是一步浅显的基于用户约定的优化：我们明确了各类 Tensor 的各类数据的初始化责任，从而避免了不必要的初始化开销。具体地：

- 用户创建 Tensor 的 `data` 由构造函数负责初始化。
- 指针函数层，总对 `forward` 的输出 `data` 初始化，但是不对 `backward` 的输出 `grad` 初始化。这里我们模仿 PyTorch 的设计：反向梯度结果总是累加到 `grad` 上，因此不需要初始化为零。
- 优化器负责初始化参数 Tensor 的 `grad`，因为优化器总是在每次迭代前清零梯度。

3.2 异步显存分配

在本项目中，Tensor 的 `data` 和 `grad` 数据都位于 GPU 显存上，且训练过程中会频繁创建中间张量与梯度张量。如果直接使用传统的 `cudaMalloc/cudaFree`，分配与释放可能触发隐式同步（尤其在默认流/跨流场景下），导致 kernel 启动与内存管理互相阻塞，进而放大每 step 的 CPU 端开销。

为此，我们在 `tensor.cu` 中将 Tensor 的显存分配与释放改为 CUDA 的异步接口：

- `cudaMallocAsync`：在指定 stream 上提交分配请求，不强制全设备同步；
- `cudaFreeAsync`：同样以 stream 顺序提交释放操作，使释放与计算在时间线上可由 CUDA runtime 做更好的调度。

具体实现上，我们定义了一个全局分配流 `kAllocStream`（当前设置为 0，即默认 stream），并用轻量封装函数 `dev_malloc/dev_free/dev_memset/dev_memcpy_h2d` 统一管理 Tensor 构造过程中对 `data/grad` 的申请、清零与 H2D 拷贝：

- 在 `Tensor(shape, useGrad)` 构造时，对 `data` 分配并清零；当 `useGrad==true` 时再为 `grad` 额外分配并清零。
- 在 `Tensor(pybind11::array_t<float>, useGrad)` 构造时，对 `data` 分配并执行异步 H2D 拷贝；`grad` 仍按需分配。
- 释放路径通过 `std::shared_ptr<float[]>` 的自定义 deleter 触发，最终调用 `cudaFreeAsync`，从而把“对象生命周期”与“异步释放提交”对齐，避免显式的手动 free。

需要说明的是，`cudaMallocAsync/cudaFreeAsync` 依赖 CUDA 的显存池机制。在我们的实现里不显式维护用户态的“块管理器”，而是将“分配/释放的同步成本”交给 CUDA runtime 的 pool 来削减；这既能降低频繁 small allocation 的开销，也能减少因 `cudaMalloc` 引起的同步。

3.3 Tensor 缓存池

除了 Tensor 自身的 `data/grad` 分配外，许多算子在前向/反向中还需要额外的临时缓冲区。典型例子包括：

- 卷积的 `img2col` 展开缓冲区（`Xhat`）；
- BatchNorm 反向传播中的临时 `dXcur/dGcur/dBcur`；
- softmax / softmax+celoss 的中间量（`rowmx/rowsum/x_y`）；
- `channelDot` 这类需要额外 tmp buffer 的融合计算。

如果对这些缓冲区每次调用都 `cudaMalloc/cudaFree`，其开销会在训练循环中被放大，并成为“算子本身已经很快，但整体仍慢”的主要原因之一。因此我们实现了一个非常轻量的“Tensor 缓存池/缓存缓冲区”机制：用静态的缓存对象保存一段可复用的 device buffer，按需扩容但尽量不缩容，从而把频繁的分配/释放变为“首次分配 + 之后复用”。

实现分布在两个位置：

1. `cudabase.cu` 中的 `CachedDeviceBuf`：
   - 维护 `ptr` 与容量 `cap`（以 `float` 元素数计）。
   - `get(n)` 在容量不足时重新分配更大的 device buffer；容量足够则直接返回现有指针。
   - 典型缓存包括：`g_ones`（用于 `colSum/rowSum` 的 ones 向量）与 `g_channelDot_tmp`（用于 `channelDot` 的逐元素乘临时区）。
2. `functional.cu` 中的 `CachedDeviceBuf`（按字节计的版本）：
   - 以 `get_bytes(bytes)` 的形式服务算子级缓存。
   - 典型缓存包括：`g_conv2d_xhat`、`g_bn_dxcur/g_bn_dgcur/g_bn_dbcur/g_bn_varcorr`、`g_softmax_rowmx/g_celoss_tmp`、以及 `g_smce_rowmx/g_smce_rowsum/g_smce_xy` 等。

这种设计的关键点在于：缓存对象的生命周期与进程一致，训练过程中反复调用算子时不会触发额外的显存分配；同时缓冲区只在“遇到更大 batch/更大特征图”时扩容一次，能很好地适配深度学习训练“shape 基本固定”的特点。

与 3.2 节的异步分配结合后，本项目的内存管理策略可以理解为“两层池化”：一层是 CUDA runtime 的 async mempool 用于 Tensor 的 `data/grad` 分配，另一层是我们在算子内部对缓冲区的显式缓存，用于避免临时缓冲区的反复申请与释放，从而提升训练吞吐并降低 CPU 端开销。

4 实验与数据分析

4.1 实验设计思路

我们的 MyTensor module 在实现时已经尽可能保持了和 PyTorch 的一致性，包括张量操作接口、计算图构建与自动微分机制、优化器设计等。因此，我们的实验设计思路是：

- 在 MyTensor 中实现、测试并调整模型结构、训练超参数；
- 在 PyTorch 中逐句翻译训练脚本，确保两者的实现逻辑完全一致；
- 方便起见，直接使用 `print` 等输出语句记录训练日志，而不使用更复杂的日志工具。

4.2 Cifar10

方便起见，我们使用了 `torchvision` 中的 `transforms` 和 `torch` 中的 `DataLoader` 来进行数据预处理和加载，并对训练集进行了简单的数据增强（随机裁剪与水平翻转）和归一化（测试集则采用与训练集相同的归一化参数）。数据预处理和加载的代码与 PyTorch 版本完全一致。

经过尝试，我们使用如下 VGG-like CNN 模型来应对 Cifar10 分类任务：

```
    Conv2D(3, 64, 3) → ReLU → BatchNorm(64)
→   Conv2D(64, 64, 3) → ReLU → BatchNorm(64)
→   MaxPool(2, 2) → Dropout(0.1)
→   Conv2D(64, 128, 3) → ReLU → BatchNorm(128)
→   Conv2D(128, 128, 3) → ReLU → BatchNorm(128)
→   MaxPool(2, 2) → Dropout(0.1)
→   Conv2D(128, 256, 3) → ReLU → BatchNorm(256)
→   Conv2D(256, 256, 3) → ReLU → BatchNorm(256)
→   MaxPool(2, 2) → Dropout(0.2)
→   Linear(4 * 4 * 256, 10)
```

其中，我们只是用了一层全连接层完成分类，这是因为实验表明此时的参数量已经足够大，再添加全连接层只会更快引发过拟合，无法提高测试集准确率。在固定这一架构的前提下，我们调试得到 Dropout 的比例，这时过拟合得到一定缓解且模型仍然能够较快地收敛。其他超参数也是在多次实验中调试得到的。

此后，我们将这一模型使用 PyTorch 复现并完成对比测试。有些出乎意料的是，在这个模型上我们的 MyTensor 实现效率略超过了 PyTorch 实现，可能的原因包括：

- 我们实现的卷积算子使用了比较朴素的 `im2col + gemm` 的方式，而 PyTorch 可能使用了更复杂的 Winograd 或 FFT 等算法，这些算法在小尺寸输入上并不一定更快；
- 我们的显存管理策略（见 3.2、3.3 节）有效地减少了内存分配的同步开销，但事实上引入了额外的且不会主动释放的缓存池；PyTorch 可能为了通用性没有采取如此激进的策略，从而在特定任务上表现不如我们。

实验结果请见 1 节表格和附录 A.2、A.3 节。

我们的模型在 10 个 epochs 的训练后即达到了约 88.4% 的测试集准确率，而在 30 个 epochs 后达到了约 89.9% 的测试集准确率。考虑到 Cifar10 的图像分辨率较低且类别数较少，这一结果已经比较令人满意，且训练效果和训练效率均能与 PyTorch 基本持平。

不足方面，在 10 个 epochs 之后，通过对比训练集表现和测试集表现，可以看出模型仍然存在一定程度的过拟合现象。 虽然我们使用了 BatchNorm 和 Dropout 等正则化手段，但仍然无法完全缓解过拟合，因而无法充分发挥模型的表达能力。可能的改进手段包括：

- 引入更多数据增强手段。
- 使用如 ResNet 等其他模型。

笔者认为，MyTensor 中的“梯度累加而不覆盖”的策略其实非常利好 ResNet 等非线性网络结构的实现，因为这些网络中存在大量的跳跃连接，若每次反向传播都覆盖梯度，则需要在计算图中保存更多的中间变量，从而增加内存开销。而梯度累加则可以避免这一问题，使得 ResNet 等网络的实现更加高效。遗憾时间有限，笔者没有进行这方面的实际尝试。

4.3 Tiny ImageNet

由于 Tiny ImageNet 数据集格式与 Cifar10 有所不同，我们自行实现了数据预处理和加载代码。我们对训练集进行了简单的数据增强（随机裁剪与水平翻转）和归一化（测试集则采用与训练集相同的归一化参数）。数据预处理和加载的代码与 PyTorch 版本完全一致。

经过尝试，我们使用如下 VGG-like CNN 模型来应对 Tiny ImageNet 分类任务：

```
    Conv2D(3, 64, 3) → ReLU → BatchNorm(64)
→   Conv2D(64, 64, 3) → ReLU → BatchNorm(64)
→   MaxPool(2, 2)
→   Conv2D(64, 128, 3) → ReLU → BatchNorm(128)
→   Conv2D(128, 128, 3) → ReLU → BatchNorm(128)
→   MaxPool(2, 2)
→   Conv2D(128, 256, 3) → ReLU → BatchNorm(256)
→   Conv2D(256, 256, 3) → ReLU → BatchNorm(256)
→   MaxPool(2, 2)
→   Conv2D(256, 512, 3) → ReLU → BatchNorm(512)
→   Conv2D(512, 512, 3) → ReLU → BatchNorm(512)
→   MaxPool(2, 2)
→   Linear(4 * 4 * 512, 1024) → ReLU
→   Linear(1024, 200)
```

相较 Cifar10，我们加深了网络并增加了全连接层，以应对 Tiny ImageNet 更复杂的分类任务。其他超参数同样是在多次实验中调试得到的。值得一提的是，我们完全放弃了 Dropout，因为模型的复杂性对于这个任务来说并不足够强大，主要的性能瓶颈仍然是欠拟合而非过拟合，即使引入极弱的 Dropout（例如 0.05 或 0.1 的比例）也会导致验证集准确率下降，且模型收敛缓慢。

实验结果请见 1 节表格和附录 A.4、A.5 节。

我们的模型在 19 个 epochs 的训练后即达到了约 40.03% 的验证集准确率，而在 36 个 epochs 后达到了 41.37% 的验证集准确率。考虑到我们实现的算子有限且算力有限，这一结果可以接受。我们的训练效果仍然能与 PyTorch 基本持平，这进一步验证的 MyTensor 实现的正确性。然而但训练效率有比较明显的落后，这主要是因为 Tiny ImageNet 的图像分辨率较高且类别数较多，导致模型参数量和计算量大幅增加，而我们实现的卷积算子并未针对大尺寸输入进行优化，因而在这一任务上表现不如 PyTorch。

不足方面，我们的算子功能仍然较少，且效率还有待进一步优化。可能的改进手段包括：

- 实现更多高效的卷积算法，如 Winograd、FFT 等；
- 优化现有算子的 CUDA 实现，提升计算效率（例如使用更高效的 reduce 算法计算 sum）。

5 总结

通过本次任务，我们实现了一个基本的深度学习框架 MyTensor，并在 Cifar10 和 Tiny ImageNet 分类任务上进行了测试。实验结果表明，我们的实现能够在训练效果与 PyTorch 基本持平（相对误差不超过 5%），在小规模任务上训练效率甚至略优于 PyTorch（当然这不意味着此时我们的框架就更加优秀，相关讨论请见 4.2 节），而在大规模任务上仍有一定差距（大约是 PyTorch 耗时的 1.6 倍）。

最终在此框架下，我们通过 VGG-like CNN 模型在 Cifar10 上达到了 89.87% 的测试集准确率，在 Tiny ImageNet 上达到了 41.37% 的验证集准确率。

A 附录

A.1 实验细节与复现方式

- 文件结构：

```
/
├── data/                       # 存放 Cifar10 和 Tiny ImageNet 数据集
└── Task3/
    ├── build/                  # CMake 构建文件夹，存放编译出的 MyTensor module
    ├── pybind11/               # pybind11 的 git 仓库
    ├── src/                    # MyTensor 源代码
    |   ├── cudabase.cu
    |   ├── cudabase.cuh        # CUDA 层与核函数层（见 2.2.4、2.2.5 节）
    |   ├── functional.h        # 张量函数层与指针函数层（见 2.2.2、2.2.3 节）
    |   ├── operator.cpp
    |   ├── operator.h          # 算子类层（见 2.2.2 节）
    |   ├── pybind.cpp
    |   ├── pybind.h            # pybind11 接口
    |   ├── tensor.cu
    |   └── tensor.h            # Tensor 类与 Function 类（见 2.1 节）
    ├── cifar10_torch.py        # 使用 PyTorch 实现的 Cifar10 训练脚本
    ├── cifar10.py              # 使用 MyTensor 实现的 Cifar10 训练脚本
    ├── CMakeLists.txt          # CMake 构建配置文件
    ├── nnmodel.py              # Python 层算子、优化器、训练与测评函数封装
    ├── tiny_imagenet_torch.py  # 使用 PyTorch 实现的 Tiny ImageNet 训练脚本
    └── tiny_imagenet.py        # 使用 MyTensor 实现的 Tiny ImageNet 训练脚本
```

- 编译 MyTensor 模块：

```bash
cd Task3
mkdir build && cd build && cmake ..
cmake --build . -j4
```

- 运行训练脚本（选择对应脚本名即可）：

```bash
cd Task3
python cifar10.py
python cifar10_torch.py
python tiny_imagenet.py
python tiny_imagenet_torch.py
```

- 提示：可以在调用 `train` 函数时设置 `verbose=True` 来打印 epoch 内部进度。

A.2 Cifar10（MyTensor）

- 超参数：
  - $epochs = 30$；
  - $lr = 0.001$，每四轮进行指数衰减 $lr \leftarrow lr \times 0.4$；
  - $\beta_{1,2} = (0.9, 0.999)$（Adam 优化器参数）；
  - $\epsilon = 10^{-8}$（Adam 优化器中的分母平滑）；

- $weight_decay = 10^{-4}$（Adam 正则化参数）；
- $seed = 712$，固定种子便于复现结果。

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
0	1.40925	0.49926	1.01905	0.35640	8.83
1	0.92809	0.32508	0.92493	0.31350	8.88
2	0.75200	0.26286	0.66427	0.22730	9.46
3	0.65411	0.22498	0.67783	0.23070	9.24
4	0.51337	0.17732	0.47510	0.16640	9.21
5	0.47606	0.16510	0.42902	0.14830	8.85
6	0.43989	0.15102	0.41688	0.14150	8.83
7	0.41956	0.14462	0.40268	0.13620	9.06
8	0.35373	0.12188	0.36037	0.12090	8.84
9	0.33486	0.11526	0.34817	0.11600	8.91
10	0.32107	0.11302	0.33745	0.11450	8.83
11	0.30968	0.10750	0.33759	0.11170	9.06
12	0.28208	0.09748	0.31801	0.10610	8.98
13	0.27235	0.09446	0.32201	0.10760	8.74
14	0.27096	0.09416	0.31988	0.10540	8.85
15	0.25984	0.08866	0.32213	0.10610	9.48
16	0.24591	0.08670	0.31377	0.10470	9.38
17	0.24805	0.08572	0.31336	0.10360	9.26
18	0.24637	0.08724	0.31218	0.10250	9.22
19	0.23972	0.08398	**0.30915**	0.10230	9.10
20	0.23715	0.08348	0.31864	0.10470	8.87
21	0.23189	0.08050	0.31325	0.10310	9.01
22	0.23605	0.08256	0.31049	0.10230	8.99
23	0.22948	0.08016	0.31187	0.10250	9.01
24	0.23033	0.08008	0.30976	0.10310	8.97
25	0.23018	0.08070	0.31209	0.10270	9.06
26	0.23087	0.08064	0.31435	0.10280	9.14
27	0.22989	**0.07998**	0.31235	**0.10210**	9.02

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
28	0.23051	0.08008	0.31420	0.10240	8.83
29	**0.22890**	0.08018	0.31030	0.10210	8.85
统计	0.22890 (min)	0.07998 (min)	0.31030 (min)	0.10210 (min)	9.03 (avg)

A.3 Cifar10（PyTorch）

- 超参数：同上。

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
0	1.42723	0.50160	1.07285	0.36550	10.36
1	0.93507	0.32614	1.08887	0.34590	9.73
2	0.75764	0.26448	0.67156	0.23520	9.53
3	0.65498	0.22800	0.61556	0.20240	10.00
4	0.51480	0.17902	0.46979	0.16240	9.89
5	0.47396	0.16516	0.42961	0.14410	9.77
6	0.44314	0.15554	0.42506	0.14230	9.55
7	0.41716	0.14436	0.40938	0.13780	9.68
8	0.35110	0.12252	0.35496	0.11920	9.58
9	0.33651	0.11660	0.35408	0.11690	9.83
10	0.32061	0.11090	0.35093	0.11910	9.54
11	0.30699	0.10576	0.34827	0.11400	9.59
12	0.28063	0.09750	0.31908	0.10620	9.88
13	0.27473	0.09420	0.31614	0.10730	9.65
14	0.27264	0.09378	0.31394	0.10300	9.66
15	0.26404	0.09216	0.31449	0.10300	9.76
16	0.24952	0.08732	0.31189	0.10340	9.59
17	0.24876	0.08786	0.30699	0.10100	9.47
18	0.24563	0.08512	0.30750	0.10090	9.55
19	0.24364	0.08448	0.30588	0.10150	9.94
20	0.23962	0.08314	0.30730	0.10150	9.70
21	0.23661	0.08264	0.30460	0.09930	9.56
22	0.23900	0.08396	0.30261	0.09950	9.61

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
23	0.23468	0.08338	**0.30240**	0.09930	9.76
24	0.23225	**0.08016**	0.30455	0.10040	9.69
25	0.23636	0.08254	0.30384	0.10110	9.57
26	0.23176	0.08064	0.30795	0.10090	9.78
27	0.23326	0.08196	0.30377	0.09960	9.62
28	**0.23140**	0.08152	0.30472	**0.09860**	9.65
29	0.23262	0.08118	0.30266	0.09890	9.53
统计	0.23140 (min)	0.08016 (min)	0.30240 (min)	0.09860 (min)	9.70 (avg)

A.4 Tiny ImageNet（MyTensor）

- 超参数：
  - $epochs = 50$；
  - $lr = 0.001$，每四轮进行指数衰减 $lr \leftarrow lr \times 0.4$；
  - $\beta_{1,2} = (0.9, 0.999)$（Adam 优化器参数）；
  - $\epsilon = 10^{-8}$（Adam 优化器中的分母平滑）；
  - $weight_decay = 10^{-4}$（Adam 正则化参数）；
  - $seed = 712$，固定种子便于复现结果。

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
0	5.04082	0.97788	5.17741	0.96980	48.98
1	4.32146	0.92417	4.09192	0.90030	49.04
2	3.92051	0.87349	3.69379	0.84260	49.36
3	3.63918	0.82713	3.52792	0.81080	49.08
4	3.33442	0.77261	3.22152	0.75580	48.68
5	3.21855	0.75314	3.17012	0.74370	49.36
6	3.12827	0.73568	3.06020	0.72020	49.07
7	3.04458	0.71963	2.98608	0.70870	48.81
8	2.88452	0.68583	2.86323	0.68390	49.17
9	2.81171	0.67270	2.80492	0.66850	48.85
10	2.73723	0.65762	2.73416	0.65610	49.61
11	2.65410	0.64079	2.72131	0.64730	49.10
12	2.54479	0.61755	2.62957	0.62930	48.74

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
13	2.49958	0.60784	2.59801	0.62050	49.28
14	2.46791	0.60064	2.57219	0.61840	48.99
15	2.42616	0.59257	2.54414	0.61190	48.69
16	2.37076	0.58020	2.52734	0.60710	49.22
17	2.35187	0.57585	2.51490	0.60450	49.13
18	2.33305	0.57231	2.48897	0.59970	48.65
19	2.31644	0.56834	2.48206	0.59770	49.43
20	2.28827	0.56235	2.46863	0.59860	49.10
21	2.28445	0.56098	2.46070	0.59420	48.94
22	2.27084	0.55795	2.46502	0.59360	49.27
23	2.26836	0.55789	2.45641	0.59100	48.99
24	2.25801	0.55504	2.45237	0.58910	48.76
25	2.24927	0.55494	2.45185	0.59210	49.46
26	2.24840	0.55381	2.44623	0.58980	49.54
27	2.24701	0.55189	2.43989	0.59020	48.91
28	2.24382	0.55281	2.44248	0.58730	49.32
29	2.23851	0.55338	2.44078	0.58750	48.96
30	2.23665	0.55159	2.44152	0.58760	49.02
31	2.23658	0.55030	2.44117	0.58800	49.52
32	2.23521	0.55141	2.44034	0.58860	49.20
33	2.23429	0.55017	2.43865	0.58710	49.19
34	2.23113	0.55162	2.44141	0.58940	48.67
35	2.23449	0.55108	2.43680	**0.58630**	49.42
36	2.23411	0.55106	2.43656	0.58670	49.23
37	2.23256	0.55031	2.44107	0.58630	49.18
38	2.23438	0.54991	2.43984	0.58690	48.91
39	2.23215	0.54822	2.43715	0.58790	48.87
40	**2.22755**	0.54811	2.43595	0.58710	48.92
41	2.22970	0.54817	2.43798	0.58710	49.26
42	2.22973	**0.54773**	2.43820	0.58850	49.19

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
43	2.22923	0.54959	2.43553	0.58870	49.11
44	2.22963	0.55047	2.43612	0.58740	48.78
45	2.23397	0.55087	**2.43513**	0.58730	49.05
46	2.22869	0.54916	2.44276	0.58880	48.94
47	2.23079	0.54925	2.43694	0.58870	48.83
48	2.23624	0.55170	2.43790	0.58960	48.93
49	2.23188	0.55015	2.43745	0.58740	48.57
统计	2.22755 (min)	0.54773 (min)	2.43513 (min)	0.58630 (min)	49.07 (avg)

A.5 Tiny ImageNet（PyTorch）

- 超参数：同上。

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
0	4.98126	0.97609	5.04692	0.96420	31.46
1	4.26423	0.91961	4.10558	0.89990	30.07
2	3.84480	0.86151	3.61920	0.82920	29.62
3	3.59410	0.82065	3.51637	0.80300	30.04
4	3.29833	0.76613	3.19097	0.75270	29.80
5	3.18568	0.74666	3.17208	0.74360	30.12
6	3.09561	0.72631	3.03442	0.71850	29.76
7	3.01463	0.71302	3.02275	0.71490	29.75
8	2.84410	0.67819	2.82688	0.67600	30.07
9	2.75861	0.66198	2.75776	0.65980	30.37
10	2.67931	0.64539	2.72974	0.64940	30.05
11	2.59551	0.62754	2.67808	0.63820	30.59
12	2.48042	0.60330	2.57684	0.61460	30.00
13	2.43416	0.59320	2.56034	0.61300	30.24
14	2.39982	0.58625	2.51639	0.60170	30.36
15	2.36021	0.57847	2.49969	0.60200	30.39
16	2.30127	0.56638	2.47651	0.59500	29.93

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
17	2.28029	0.56283	2.46528	0.59180	29.93
18	2.26109	0.55655	2.43708	0.58890	30.22
19	2.24401	0.55400	2.43254	0.58530	29.69
20	2.21663	0.54721	2.41847	0.58150	30.41
21	2.21251	0.54574	2.41219	0.57880	31.13
22	2.20076	0.54285	2.41417	0.57810	30.56
23	2.19694	0.54306	2.40707	0.57960	30.80
24	2.18214	0.53919	2.40404	0.57910	29.83
25	2.17948	0.53844	2.40456	0.57880	30.50
26	2.17459	0.53910	2.39934	0.57580	29.93
27	2.17306	0.53678	2.39128	0.57770	30.21
28	2.16881	0.53820	2.39393	0.57540	30.08
29	2.16901	0.53654	2.39151	0.57550	30.32
30	2.16501	0.53615	2.39547	0.57450	30.68
31	2.16255	0.53516	2.39235	0.57690	30.24
32	2.16422	0.53534	2.39104	0.57400	30.35
33	2.16271	0.53592	2.39162	0.57500	30.32
34	2.15864	0.53498	2.39223	0.57600	30.79
35	2.15788	0.53558	2.38829	0.57590	30.74
36	2.16229	0.53479	2.38858	0.57480	30.95
37	2.15602	0.53412	2.39185	0.57520	29.80
38	2.16078	0.53540	2.39213	0.57340	30.26
39	2.15544	**0.53209**	2.39100	0.57560	30.10
40	**2.15513**	0.53562	2.38723	0.57370	30.20
41	2.15863	0.53367	2.39155	0.57380	29.88
42	2.15681	0.53364	2.39247	0.57560	29.65
43	2.15869	0.53447	2.38842	0.57530	29.72
44	2.15680	0.53478	2.38875	0.57540	29.61
45	2.15983	0.53530	**2.38673**	**0.57310**	29.50
46	2.15797	0.53545	2.39514	0.57360	30.27

Epoch	Train Loss	Train Error	Test Loss	Test Error	Time/s
47	2.15800	0.53483	2.38876	0.57350	30.99
48	2.16050	0.53541	2.39043	0.57330	31.76
49	2.15805	0.53582	2.38957	0.57330	29.62
统计	2.15513 (min)	0.53209 (min)	2.38673 (min)	0.57310 (min)	30.23 (avg)